

MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Project
Software Design

Author: Gligor Sorana-Maria
Group: 30433

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

June 2026

Contents

1	Problem Definition	2
2	Analysis Phase - Use Cases & Requirements	3
2.1	Functional Requirements	3
2.2	Use Cases	4
3	Design Phase	7
3.1	Architecture Overview	7
3.2	Entity-Relationship Diagram	8
3.3	Component: Country Service API	9
3.4	Component: React Client	10
3.5	Authentication Flow	10
3.6	Real-Time Chat	10
3.7	Internationalization	10
3.8	Error Handling	11
3.9	Class Diagram	11
3.10	Sequence Diagram	11
4	Implementation Phase - Application	13
4.1	Tools and Technologies	13
4.2	Application Structure	13
4.3	User Manual	14
5	Testing and Validation	16
5.1	Unit Testing	16
5.2	CI/CD Pipeline	16
5.3	Manual Testing	16
5.4	Validation	17

Chapter 1

Problem Definition

This document describes the evolution of the **Country Management Web Application** from a microservices architecture (Assignment 3) into a full **client-server distributed application**. The domain remains unchanged: the application manages country data and supports three user roles — Visitor, User and Administrator.

The goal of this assignment is to implement a clear separation between a client application and a server-side API, introduce real-time communication via WebSockets, provide a multilingual user interface and implement graceful error handling on both the client and server sides.

The system consists of four independent components:

- **Country Service** (port 8080): a RESTful API server with WebSocket support for chat. It no longer serves HTML pages — all UI is handled by the client.
- **User Service** (port 8081): manages user accounts, credentials, roles and visited country records. Exposes internal REST endpoints.
- **Notification Service** (port 8082): receives event notifications via REST and sends email notifications.
- **React Client** (port 3000): a single-page application (SPA) that communicates with the Country Service via REST and WebSocket. Supports five languages and handles errors gracefully.

The deliverables include the source code of all four components in a public Git repository and this documentation report.

Chapter 2

Analysis Phase - Use Cases & Requirements

2.1 Functional Requirements

All functional requirements from Assignments 1, 2 and 3 are retained. The following new requirements are introduced for the Project:

1. The client must be a separate application that communicates with the server exclusively through its exposed API endpoints.
2. The server must expose a RESTful API for all CRUD operations, queries, exports and authentication.
3. The system must support real-time chat using WebSockets. Any user (including unauthenticated visitors) can send messages. Authenticated users may optionally post as Anonymous.
4. Chat messages must display the sender's name and timestamp and must appear without refreshing the page.
5. Chat history must persist across sessions using browser local storage.
6. The user interface must support at least two languages and allow dynamic language switching. The implemented languages are English, Romanian, French, German and Hungarian.
7. Server-side errors must be returned as structured JSON objects containing a message, error code and timestamp.
8. Client-side errors must be displayed in a user-friendly manner without crashing the application.
9. Authentication must use JWT tokens. The client stores the token in local storage and sends it in the **Authorization** header.

2.2 Use Cases

The use cases from previous assignments remain unchanged from the user's perspective. New use cases are added for chat and language switching.

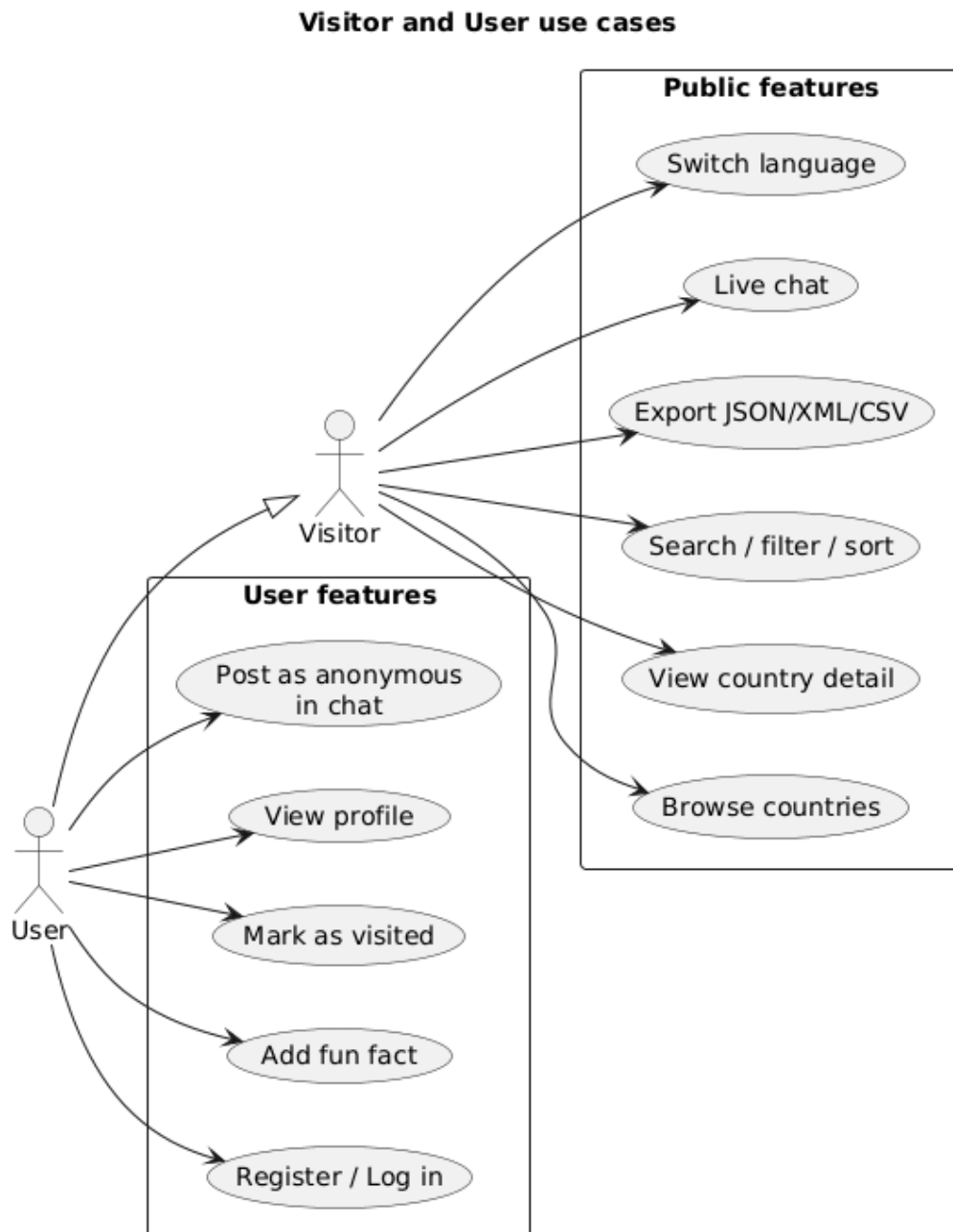


Figure 2.1: Use Case Diagram: Visitor and User

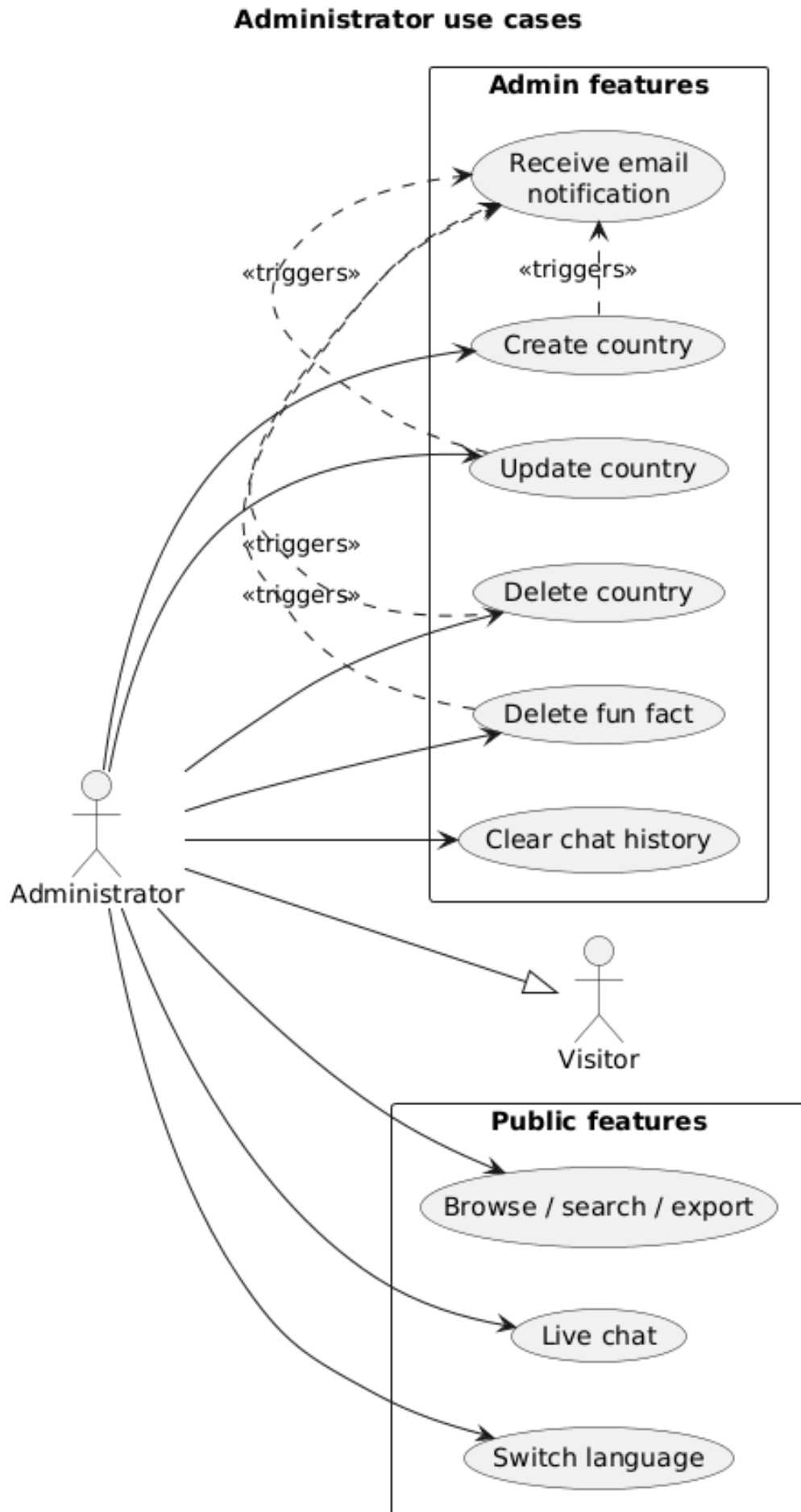


Figure 2.2: Use Case Diagram: Administrator

From the diagram, the Visitor actor can browse, search, filter, sort and export countries, view country details, use the live chat and switch the interface language. The User actor inherits all Visitor use cases and can additionally register, log in, add fun facts, mark countries as visited and view their profile. The Administrator actor inherits all Visitor use cases and can create, update and delete countries, delete fun facts and clear the chat history.

Chapter 3

Design Phase

3.1 Architecture Overview

The system follows a **client-server architecture** where the React client runs independently in the browser and communicates with the Country Service via two protocols:

- **REST HTTP**: for all standard operations — authentication, country queries, CRUD, profile and export.
- **WebSocket (STOMP over SockJS)**: for the real-time chat feature. The client connects to `/ws/chat`, subscribes to `/topic/chat`, and publishes messages to `/app/chat.send`.

The Country Service communicates with the User Service and Notification Service via synchronous REST calls, exactly as in Assignment 3. Authentication is stateless — the server issues JWT tokens on login and validates them on every request using a `JwtFilter` in the Spring Security filter chain.

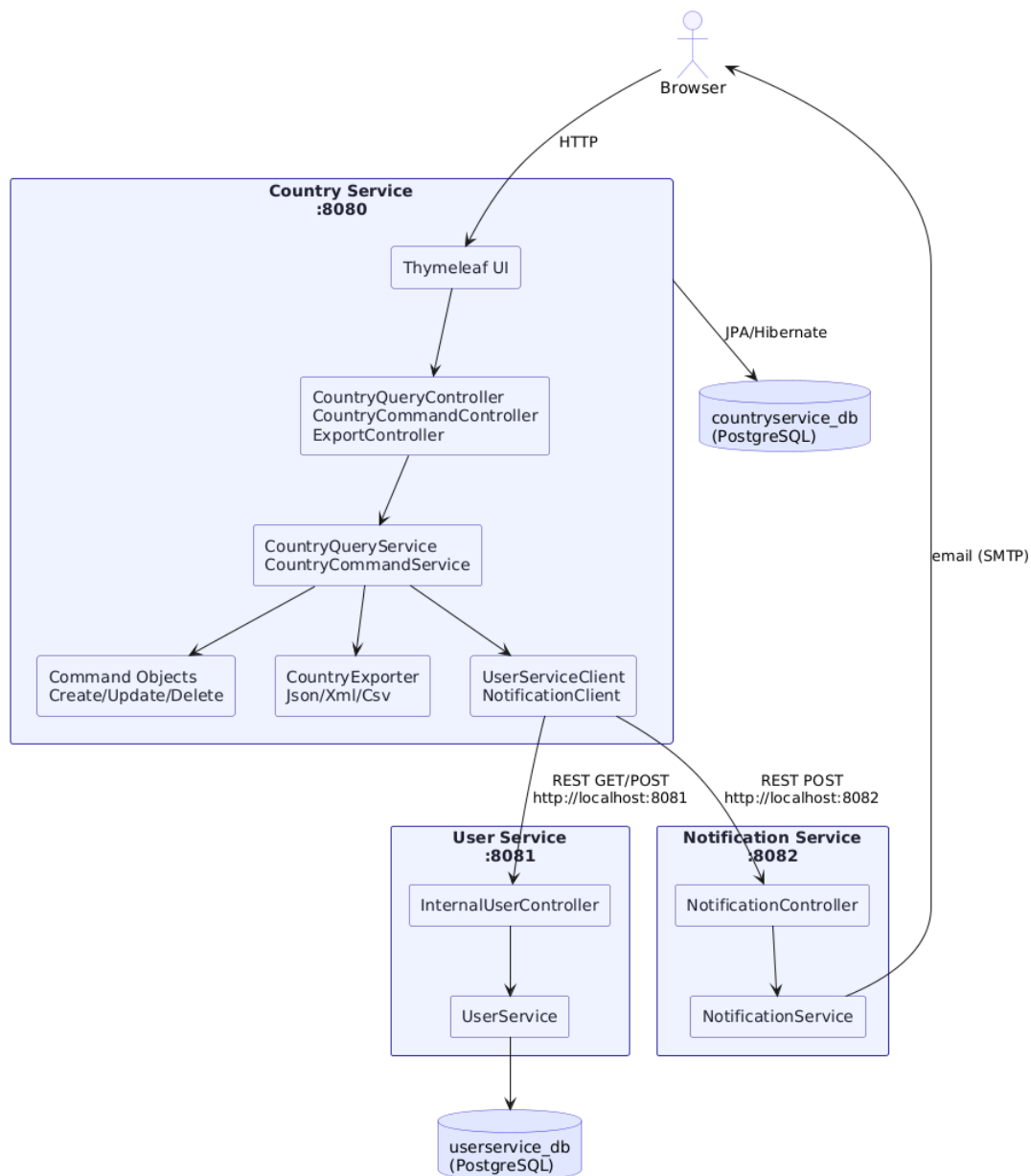


Figure 3.1: System Architecture Overview

3.2 Entity-Relationship Diagram

The database schema is unchanged from Assignment 3. Two PostgreSQL databases are used: **countryservice_db** (countries, fun_facts) and **userservice_db** (users, visited_countries).

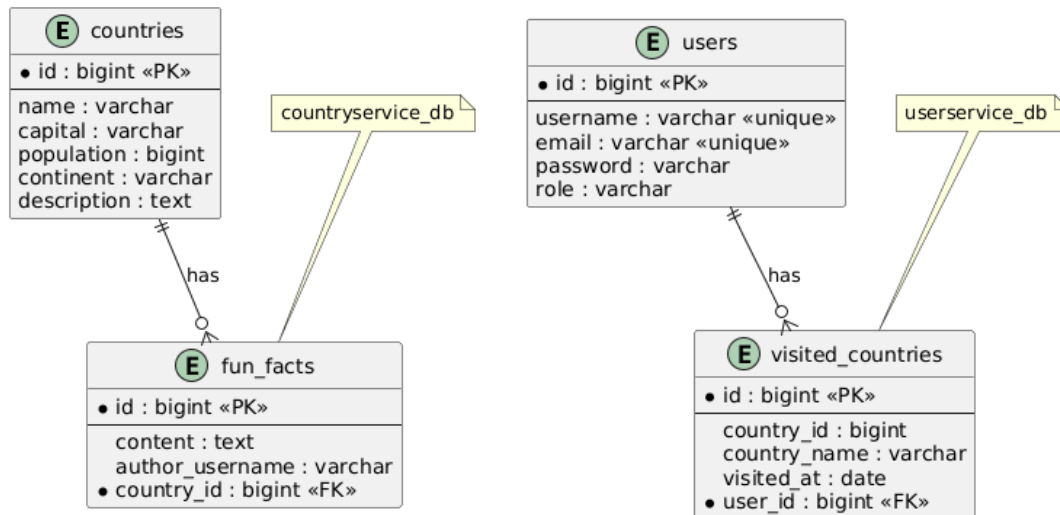


Figure 3.2: Entity-Relationship Diagram

3.3 Component: Country Service API

The Country Service no longer contains any Thymeleaf templates. It exposes the following REST endpoints:

- **POST** `/api/auth/login`: validates credentials via User Service, returns a JWT token.
- **POST** `/api/auth/register`: forwards registration to User Service.
- **GET** `/api/countries`: paginated, filtered and sorted country list.
- **GET** `/api/countries/{id}`: single country with fun facts.
- **POST** `/api/countries/{id}/fun-facts`: add a fun fact (USER, ADMIN).
- **DELETE** `/api/countries/{id}/fun-facts/{factId}`: delete a fun fact (ADMIN).
- **POST** `/api/admin/countries`: create country (ADMIN).
- **PUT** `/api/admin/countries/{id}`: update country (ADMIN).
- **DELETE** `/api/admin/countries/{id}`: delete country (ADMIN).
- **GET** `/api/countries/export`: download JSON, XML or CSV with active filters.
- **GET** `/api/profile/visited`: visited countries for the logged-in user.
- **POST** `/api/profile/visited/{id}`: mark a country as visited.
- **GET** `/api/profile/visited/{id}/check`: check if a country is visited.
- **WebSocket** `/ws/chat`: STOMP endpoint for real-time chat.

All error responses follow a consistent structure containing `errorCode`, `message` and `timestamp`, implemented via a `GlobalExceptionHandler` annotated with `@RestControllerAdvice`.

3.4 Component: React Client

The React client is a TypeScript single-page application built with Create React App. It is organized into the following modules:

- **api/axios.ts**: Axios instance with base URL, JWT interceptor for outgoing requests and error interceptor for consistent error handling.
- **context/AuthContext.tsx**: React context storing the JWT token, username and role in local storage. Provides `login()` and `logout()` methods.
- **i18n/**: Translation files for English, Romanian, French, German and Hungarian. Language is persisted in local storage and switched dynamically via a dropdown in the navigation bar.
- **components/Navbar.tsx**: navigation bar with role-based links, language selector and logout button.
- **components/Chat.tsx**: WebSocket chat using `@stomp/stompjs` and `SockJS`. Messages are persisted in local storage. Authenticated users may post as Anonymous. Administrators can clear the history.
- **components/ErrorAlert.tsx**: reusable dismissible error banner shown when API calls fail.
- **pages/**: `CountryList`, `CountryDetail`, `Login`, `Register`, `Profile`, `AdminDashboard` — each page uses the `api` module for data fetching and `ErrorAlert` for error display.

3.5 Authentication Flow

Authentication is stateless and JWT-based. On login the client sends credentials to `POST /api/auth/login`. The Country Service calls the User Service to verify the password using `BCrypt`, then generates a signed JWT containing the username and role. The client stores this token in local storage and attaches it to every subsequent request via the Axios interceptor. The Spring Security `JwtFilter` validates the token on every request and populates the `SecurityContext`.

3.6 Real-Time Chat

The chat uses STOMP over SockJS. On mount, `Chat.tsx` creates a `Client` instance connected to `http://localhost:8080/ws/chat`. Once connected, it subscribes to `/topic/chat`. When a message is published to `/app/chat.send`, `ChatController` on the server broadcasts it to all subscribers via `SimpMessagingTemplate`. Messages appear instantly for all connected clients without page refresh. The connection status is shown as a colored badge. Messages are saved to `localStorage` so history survives page reloads and logouts.

3.7 Internationalization

The `i18next` library with `react-i18next` provides translation support. Translation keys are defined in five JSON files: `en.json`, `ro.json`, `fr.json`, `de.json` and `hu.json`. The

selected language is stored in `localStorage` and applied on startup. A dropdown in the navigation bar allows users to switch languages dynamically at any time without reloading the page.

3.8 Error Handling

Server side: a `GlobalExceptionHandler` annotated with `@RestControllerAdvice` catches `RuntimeException`, `AccessDeniedException` and generic `Exception` and returns a structured `ErrorResponse` object with `errorCode`, `message` and `timestamp`. Internal implementation details are never exposed to the client.

Client side: the Axios response interceptor transforms all error responses into a consistent `{ message, errorCode, status }` object. Each page catches errors in `try/catch` blocks and passes the message to the `ErrorAlert` component, which displays a dismissible red banner. The application never crashes on API failure — it always shows a meaningful message to the user.

3.9 Class Diagram

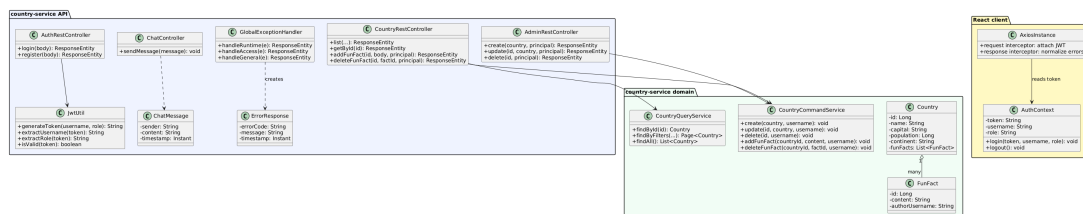


Figure 3.3: Core Domain and API Class Diagram

3.10 Sequence Diagram

The most important workflow involving all system components is user login followed by marking a country as visited.

1. The user submits the login form. React sends `POST /api/auth/login` with credentials.
2. `AuthRestController` calls `UserServiceClient.checkPassword()`.
3. `UserServiceClient` sends `POST /internal/users/check-password` to User Service.
4. User Service verifies the BCrypt hash and returns `true`.
5. `AuthRestController` calls `JwtUtil.generateToken()` and returns the JWT to the client.
6. The client stores the token in `localStorage` and updates `AuthContext`.
7. The user navigates to a country detail page. React sends `GET /api/countries/{id}` with the JWT header.

8. `JwtFilter` validates the token and populates the `SecurityContext`.
9. `CountryRestController` returns the country with fun facts.
10. The user clicks Mark as visited. React sends `POST /api/profile/visited/{id}`.
11. `ProfileRestController` calls `UserServiceClient.markVisited()`.
12. User Service saves the `VisitedCountry` record and returns 200.
13. React updates the UI to show the visited badge without reloading.

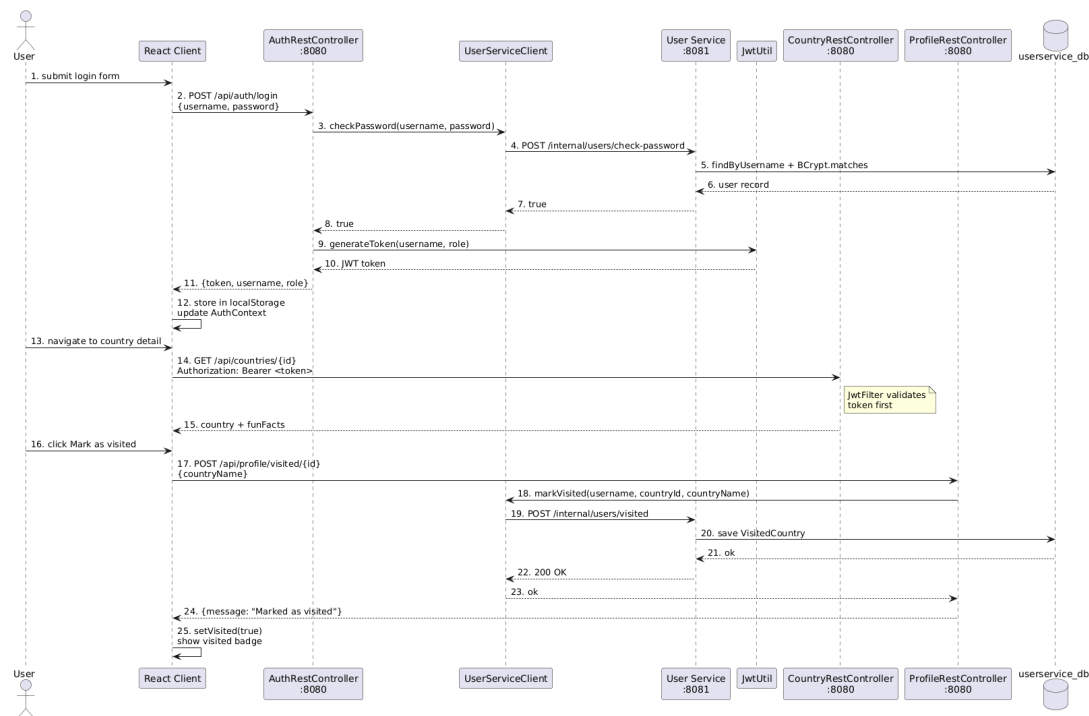


Figure 3.4: Sequence Diagram: Login and Mark Country as Visited

Chapter 4

Implementation Phase - Application

4.1 Tools and Technologies

All tools and technologies from Assignment 3 are retained. The following are added:

- **React 18 with TypeScript**: the client-side framework. TypeScript provides compile-time type safety.
- **React Router v6**: client-side routing between pages without full page reloads.
- **Axios**: HTTP client with request and response interceptors for JWT attachment and error normalization.
- **i18next + react-i18next**: internationalization framework supporting five languages with dynamic switching.
- **@stomp/stompjs + sockjs-client**: WebSocket client library for STOMP protocol over SockJS.
- **Spring WebSocket + STOMP**: server-side WebSocket support with message broker configuration.
- **JWT 0.12.6**: JWT generation and validation library used by `JwtUtil`.
- **Spring Security (stateless)**: configured with `SessionCreationPolicy.STATELESS` and a custom `JwtFilter`.

4.2 Application Structure

```
1 repo-root/  
2   country/  
3     src/main/java/com/sds2/country/  
4       config/      <- SecurityConfig, WebSocketConfig,  
5                     JwtUtil, JwtFilter,  
6                     GlobalExceptionHandler, RestConfig  
7       controller/  <- AuthRestController, CountryRestController,  
8                     AdminRestController, ProfileRestController,  
9                     ExportRestController, ChatController
```

```

10      command/          <- CountryCommand, Create/Update/
      DeleteCountryCommand
11      client/           <- UserServiceClient, NotificationClient
12      export/           <- CountryExporter, Json/XML/CsvCountryExporter
13      model/            <- Country, FunFact, ChatMessage
14      repository/       <- CountryRepository, FunFactRepository
15      service/          <- CountryQueryService, CountryCommandService,
16                          RemoteUserDetailsService
17
18      user/
19      src/main/java/com/sds2/userservice/
20      config/           <- SecurityConfig
21      controller/       <- InternalUserController
22      model/            <- User, Role, VisitedCountry
23      repository/       <- UserRepository, VisitedCountryRepository
24      service/          <- UserService
25
26      notification/
27      src/main/java/com/sds2/notificationsservice/
28      controller/       <- NotificationController
29      service/          <- NotificationService
30
31      client/
32      src/
33      api/              <- axios.ts
34      components/       <- Navbar.tsx, Chat.tsx, ErrorAlert.tsx
35      context/          <- AuthContext.tsx
36      i18n/             <- index.ts, en.json, ro.json,
37                          fr.json, de.json, hu.json
38      pages/            <- CountryList.tsx, CountryDetail.tsx,
39                          Login.tsx, Register.tsx,
40                          Profile.tsx, AdminDashboard.tsx
41
42      .github/workflows/ci.yml
43      documentation/

```

4.3 User Manual

Start all services in order: User Service (8081), Notification Service (8082), Country Service (8080), then the React client (`npm start`, port 3000). Open <http://localhost:3000>.

As a Visitor: browse and search countries, sort columns, paginate, view country details and export in JSON, XML or CSV. Use the language dropdown in the navbar to switch between English, Romanian, French, German and Hungarian. Open the Chat page to send messages — you will appear as Anonymous.

As a User: register at `/register`, log in, add fun facts to countries, mark countries as visited, view visited countries on the profile page. In the chat you can choose to post under your username or as Anonymous using the checkbox below the input.

As an Administrator: access the admin dashboard to create, edit and delete countries.

Delete fun facts from the country detail page. A Clear button appears in the chat to wipe the message history.

Chapter 5

Testing and Validation

5.1 Unit Testing

Unit tests from Assignment 3 are retained unchanged. All three services pass their test suites with `mvn test`.

5.2 CI/CD Pipeline

The GitHub Actions workflow in `.github/workflows/ci.yml` runs on every push to `main`. It has three parallel jobs — one per Java service — each setting up Java 21, caching Maven dependencies and running `mvn test`. Environment variables for database and mail credentials are passed as dummy values since unit tests use Mockito and never connect to real external services.

5.3 Manual Testing

- **Authentication:** verified that login returns a JWT token stored in local storage. Verified that accessing admin endpoints without a token returns HTTP 403 with a structured error response.
- **JWT expiry:** verified that expired tokens result in 401 responses and the client shows an appropriate error message.
- **Real-time chat:** opened two browser tabs (one logged in, one not). Verified that messages sent in one tab appear instantly in the other. Verified that anonymous users appear as "Anonymous" and logged-in users appear with their username. Verified that the post-as-anonymous checkbox works correctly.
- **Chat persistence:** verified that chat history survives page refresh and logout/login cycles via local storage. Verified that the admin-only Clear button wipes the history.
- **Internationalization:** verified that switching language in the dropdown updates all visible text instantly without page reload. Verified that the selected language persists across page navigation and browser refresh.

- **Error handling:** tested submitting invalid login credentials — server returns a structured `INVALID_CREDENTIALS` error, client displays the message in the error banner. Tested accessing a non-existent country — server returns `NOT_FOUND`, client shows the error banner. Verified that no raw stack traces or internal details are exposed.
- **Export:** verified that all three formats download correctly with active filters and sort order applied.
- **CRUD:** verified that admin create, edit and delete operations work correctly from the dashboard and that changes are immediately reflected in the country list.

5.4 Validation

All functional requirements from Assignments 1, 2, 3 and the Project are satisfied. The client-server separation is complete — the React app communicates exclusively through the REST API and WebSocket endpoint. Real-time chat works across multiple clients simultaneously. The interface supports five languages with instant switching. Errors are handled gracefully on both server and client sides with structured, user-friendly messages.